Head First Kotlin

Chapter 1 - getting started: A Quick Dip

- Note: Head First starts with the most basic of basics, but I'm a sucker for provided chapter summaries.
- Use `fun` to define a function.
- Every application needs a function named `main`.
- Use // to denote a single-lined comment.
- A `String` is a string of characters. You denote a `String` value by enclosing its characters in double quotes.
- Code blocks are defined by a pair of curly braces `{ }`.
- The assignment operator is *one* equals sign `=`.
- The equals operator is *two* equals signs `==`.
- Use `var` to define a variable whose value may change.
- Use `val` to define a value whose value will stay the same.
- A `while` loop runs everything within its block so long as the conditional test is *true*.
- If the conditional test is *false*, the `while` loop code block won't run, and execution will move down to the code immediately after the loop block.
- Put a conditional test inside parentheses `( )`.
- Add conditional branches to your code using `if` and `else`. The `else` clause is optional.
- You can use `if` as an expression so that it returns a value. In this case, the `else` clause is mandatory.

```
println(if (x > y) "x is greater than y" else "x is not greater than y")
```

Chapter 2 - basic types and variables: Being a Variable

- In order to create a variable, the compiler needs to know its name, its type, and whether it can be reused.
- If the variable's type isn't explicitly defined, the compiler infers it from its value.
- A variable holds a reference to an object.
- An object has state and behavior. Its behavior is exposed through its functions.
- Defining the variable with `var` means the variable's object reference can be replaced. Defining the variable with `val` means the variable holds a reference to the same object forever.
- Explicitly define a variable's type by putting a colon after the variable's name, followed by the type: `var tinyNum: Byte`
- You can only assign a value to a variable that has a compatible type.
- You can convert one numeric type to another. If the value won't fit into the new type, some precision is lost.
- Create an array using the `arrayOf` function: `var myArray = arrayOf(1, 2, 3)`
- Access an array's items using, for example, `myArray[0]`. The first item in an array has an index of 0.
- Get an array's size using `myArray.size`.
- The compiler infers the array's type from its items. You can explicitly define an array's type like this: `var myArray: Array<Byte>`
- If you define an array using `val`, you can still update the items in the array.
- `String` templates provide a quick and easy way of referring to a variable or evaluating an expression from inside a `String`.

Chapter 3 - functions: Getting Out of Main

- Use functions to organize your code and make it more reusable.
- A function can have parameters, so you can pass more than one value to it.
- The number and type of values you pass to the function must match the order and type of the parameters declared by the function.
- A function can return a value. You must define the type of value (if any) it returns.
- A `Unit` return type means that the function doesn't return anything.

- Choose `for` loops over `while` loops when you know how many times you want to repeat the loop code.
- The `readLine()` function reads a line of input from the standard input stream. It returns a `String` value, the text entered by the user.
- If the input stream has been redirected to a file and the end of the file has been reached, the `readLine()` function returns `null`. `null` means it has no value, or it's missing.
- `&&` means "and". `||` means "or". `!` means "not".

Chapter 4 - classes and objects: A Bit of Class

- Classes let you define your own types.
- A class is a template for an object. One class can create many objects.
- The things an object knows about itself are its properties. The things an object can do are its functions.
- A property is a variable that's local to the class.
- The `class` keyword defines a class.
- Use the dot operator to access an object's properties and functions.
- A constructor runs when you initialize an object.
- You can define a property in the primary constructor by prefixing a parameter with `val` or `var`. You can define a property outside the constructor by adding it to the class body.
- Initializer blocks run when an object is initialized.
- You must initialize each property before you use its value.
- Getters and setters let you get and set property values.
- Behind the scenes, the compiler adds a default getter and setter to every property.

Chapter 5 - subclasses and superclasses: Using Your Inheritance

- A superclass contains common properties and functions that are inherited by one or more subclasses.
- A subclass can include extra properties and functions that aren't in the superclass, and can override the things it inherits.
- Use the IS-A test to verify that your inheritance is valid. If X is a *subclass* of Y, then X IS-A Y must make sense.

- The IS-A relationship works in only one direction. A Hippo is an Animal, but not all Animals are Hippos.
- If class B is a subclass of class A, and class C is a subclass of class B, class C passes the IS-A test for both B and A.
- Before you can use a class as a superclass, you must declare it open. You must also declare any properties and functions you want to override as open.
- Use `:` to specify a subclass's superclass.
- If the superclass has a primary constructor, then you must call it in the subclass header.
- Override properties and functions in the subclass by prefixing them with `override`. When you override a property, its type must be compatible with that of the superclass property. When you override a function, its parameter list must stay the same, and its return type must be compatible with that of the superclass.
- Overridden functions and properties stay open until they're declared `final`.
- When a function is overridden in a subclass, and that function is invoked on an instance of the subclass, the overridden version of the function is called.
- Inheritance guarantees that all subclasses have the functions and properties defined in the superclass.
- You can use a subclass in any place where the superclass type is expected.
- Polymorphism means "many forms". It allows different subclasses to have different implementations of the same function.

Chapter 6 - abstract classes and interfaces: Serious Polymorphism

- An abstract class can't be instantiated. It can contain both abstract and non-abstract properties and functions.
- Any class that contains an abstract property or function must be declared abstract.
- A class that's not abstract is called concrete.
- You implement abstract properties and functions by overriding them.
- All abstract properties and functions must be overridden in any concrete subclasses.
- An interface lets you define common behavior outside a superclass hierarchy so that independent classes can still benefit from polymorphism.
- Interfaces can have abstract or non-abstract functions.

- Interface properties can be abstract, or they can have getters and setters. They can't be initialized, and they don't have access to a backing field.
- A class can implement multiple interfaces.
- If a subclass inherits from a superclass or implements an interface named A, you can use the code: `super<A>.myFunction()` to call the implementation of `myFunction` that is defined in A.
- If a variable holds a reference to an object, you can use the `is` operator to check the type of the underlying object.
- The `is` operator performs a smart cast when the compiler can guarantee that the underlying object can't have changed between the type check and its usage.
- The `as` operator lets you perform an explicit cast.
- A `when` expression lets you compare a variable against an exhaustive set of different options.

Chapter 7 - data classes : Dealing with Data

- The behavior of the `==` operator is determined by the implementation of the `equals` function.
- Every class inherits an `equals`, `hashCode`, and `toString` function from the `Any` class because every class is a subclass of `Any`. These functions can be overridden.
- The `equals` function tells you if two objects are considered "equal". By default, it returns `true` if it's used to test the same underlying object, and `false` if it's used to test separate objects.
- The `===` operator lets you check whether two variables refer to the same underlying object irrespective of the object's type.
- A data class lets you create objects whose main purpose is to store data. It automatically overrides the `equals`, `hashCode`, and `toString` functions, and includes `copy` and `componentN` functions.
- The data class `equals` function checks for equality by looking at each object's property values. If two data objects hold the same data, the `equals` function returns `true`.
- The `copy` function lets you create a new copy of a data object, altering some of its properties. The original object remains intact.

- `componentN` functions let you destructure data objects into their component property values.
- A data class generates its functions by considering the properties defined in its primary constructor.
- Constructors and functions can have default parameter values. You can call a constructor or function by passing parameter values in order of declaration or by using named arguments.
- Classes can have secondary constructors.
- An overloaded function is a different function that happens to have the same function name. An overloaded function must have different arguments, but may have a different return type.
- Rules for Data Classes
    - There must be a primary constructor.
    - The primary constructor must define one or more parameters.
    - Each parameter must be marked as `val` or `var`.
    - Data classes must not be open or abstract.

Chapter 8 - nulls and exceptions: Safe and Sound

- `null` is a value that means a variable doesn't hold a reference to an object. The variable exists, but it doesn't refer to anything.
- A nullable type can hold null values in addition to its base type. You define a type as nullable by adding a `?` to the end of it.
- To access a nullable variable's properties and functions, you must first check that it's not null.
- If the compiler can't guarantee that a variable is not null in between a null-check and its usage, you must access properties and functions using the safe call operator (`?.`).
- You can chain safe calls together.
- To execute code if (and only if) a value is not null, use `?.let`.
- The Elvis operator (`?:`) is a safe alternative to an `if` expression.
- The not-null assertion operator (`!!`) throws a `NullPointerException` if the subject of your assertion is null.

- An exception is a warning that occurs in exceptional situations. It's an object of type `Exception`.
- Use `throw` to throw an exception.
- Catch an exception using `try` / `catch` / `finally`.
- `try` and `throw` are expressions.
- Use a safe cast (`as?`) to avoid getting a `ClassCastException`.

Chapter 9 - collections: Get Organized

- Create an array initialized with `null` values use the `arrayOfNulls` function.
- Useful array functions include: `sort`, `reverse`, `contains`, `min`, `max`, `sum`, `average`.
- The Kotlin Standard Library contains pre-built classes and functions grouped into packages.
- A `List` is a collection that knows and cares about index position. It can contain duplicate values.
- A `Set` is an unordered collection that doesn't allow duplicate values.
- A `Map` is a collection that uses key/value pairs. It can contain duplicate values, but not duplicate keys.
- `List`, `Set`, and `Map` are immutable. `MutableList`, `MutableSet`, and `MutableMap` are mutable subtypes of these collections.
- Create a `List` using `listOf` function.
- Create a `MutableList` using `mutableListOf` function.
- Create a `Set` using `setOf` function.
- Create a `MutableSet` using `mutableSetOf` function.
- A `Set` checks for duplicates by first looking for matching hash code values. It then uses the `===` and `==` operators to check for referential or object equality.
- Create a `Map` using `mapOf` function.
- Create a `MutableMap` using `mutableMapOf` function.

Chapter 10 - generics: Know Your Ins from Your Outs

- Generics let you write consistent code that's type-safe. Collections such as `MutableList` use generics.
- The generic type is defined inside angle brackets `<>`, for example: `class Contest<T>`

- You can restrict the generic type to a specific supertype, for example: `class Contest<T: Pet>`
- You create an instance of a class with a generic type by specifying the "real" type in angle brackets, for example: `Contest<Cat>`
- Where possible, the compiler will infer the generic type.
- You can define a function that uses a generic type outside a class declaration, or one that uses a different generic type, for example:

```
fun <T> listPet(): List<T> {
    //...
}
```

- A generic type is invariant if it can only accept references of that specific type. Generic types are invariant by default.
- A generic type is covariant if you can use a subtype in place of a supertype. You specify that a type is covariant by prefixing it with `out`.
- A generic type is contravariant if you can use a supertype in place of a subtype. You specify that a type is contravariant by prefixing it with `in`.

Chapter 11 - lambdas and higher-order functions: Treating Code Like Data

- A lambda expression, or lambda, takes the form:

```
{ x: Int -> x + 5 }
```

The lambda is defined within curly braces, and can include parameters, and a body.

- A lambda can have multiple lines. The last evaluated expression in the body is the lambda's return value.
- You can assign a lambda to a variable. The variable's type must be compatible with the type of the lambda.
- A lambda's type has the format: `(parameters) -> return_type`
- Where possible, the compiler can infer the lambda's parameter types.
- If the lambda has a single parameter, you can replace it with `it`.

- You execute a lambda by invoking it. You do this by passing the lambda any parameters in parentheses, or by calling its `invoke` function.
- You can pass a lambda to a function as a parameter, or use one as a function's return value. A function that uses a lambda in this way is known as a higher-order function.
- If the final parameter of a function is a lambda, you can move the lambda outside the function's parentheses when you call the function.
- If a function has a single parameter that's a lambda, you can omit the parentheses when you call the function.
- A type alias lets you provide an alternative name for an existing type. You define a type alias using `typealias`.

Chapter 12 - built-in higher-order functions: Power Up Your Code

- Use `minBy` and `maxBy` to find the lowest or highest value in a collection. These functions take one parameter, a lambda whose body specifies the function criteria. The return type matches the type of items in the collection.
- Use `sumBy` or `sumByDouble` to return the sum of items in a collection. Its parameter, a lambda, specifies the thing you want to sum. If this is an `Int`, use `sumBy`, and if it's a `Double`, use `sumByDouble`.
- Note: Both `sumBy` and `sumByDouble` are deprecated. Use `sumOf` which works generically for any type.
- The `filter` function lets you search, or filter, a collection according to some criteria. You specify this criteria using a lambda, whose lambda body must return a `Boolean`. `filter` usually returns a `List`. If the function is being used with a `Map`, however, it returns a `Map` instead.
- The `map` function transforms the items in a collection according to some criteria that you specify using a lambda. It returns a `List`.
- `forEach` works like a `for` loop. It allows you to perform one or more actions for each item in a collection.
- Use `groupBy` to divide a collection into groups. It takes one parameter, a lambda, which defines how the function should group the items. The function returns a `Map`, which uses the lambda criteria for the keys, and a `List` for each value.
- The `fold` function lets you specify an initial value, and perform some operation for each item in a collection. It takes two parameters: the initial value and a lambda that specifies the

operation you want to perform.

Appendices

Appendix A - coroutines: Running Code in Parallel

- See a more recent resource on coroutines

Appendix B - testing: Hold Your Code to Account

- `junit` works with Kotlin, but there is also https://github.com/kotest/kotest
- See https://kotest.io/docs/quickstart for more about Kotlin Test!

Appendix C - leftovers: The Top Ten Things: (We Didn't Cover)

Packages

- Packages help you organize code and prevent name conflicts.
- Package declarations are like `package com.hfkotlin.mypackage`.
- From another package use `import com.hfkotlin.mypackage.Duck` to use Duck.

Visibility Modifiers

- Kotlin has `public`, `private`, `protected`, and `internal`.
- Everything is `public` by default.
- `internal` is only within the same module (a set of files compiled together).

Enum Classes

- `enum class BandMember { JERRY, BOBBY, PHIL }`
- An enum can have a constructor to initialize it with a value.
- An enum can also have methods in the main body of the class.

Sealed Classes

- A "sealed class" is like a souped up enum.

```
sealed class MessageType
class MessageSuccess(var msg: String) : MessageType()
class MessageFailure(var msg: String, var e: Exception) : MessageType()
```

- Sealed classes work in `when` blocks without `else` clauses.

Nested and inner classes

- A *nested class* is defined inside another class.
- You can refer to a `Nested` class within `Outer` like:
  `val nested = Outer.Nested()`
- A nested class doesn't have access to an instance of the outer class so cannot access its members.
- An *inner class* can access outer class members and is defined as:
  `inner class Inner`
- You create an instance of the `Outer` class to access the `Inner`:
  `val inner = Outer().Inner()`
  or by instantiating a property of the Outer class with the Inner class.
- An inner class is always tied to a specific instance of the outer class.

Object declarations and expressions

- The `object` keyword defines a class and makes a declaration in one statement. Only one instance of that type will ever be created.
- It looks like a class but cannot have a constructor.

```
object DuckManager {
    val allDucks = mutableListOf<Duck>()

    // call `DuckManager.herdDucks()` to use this functionality
    fun herdDucks() {
        // Code to herd the Ducks
    }
}
```

- An `object` can be declared within a class to provide static methods.
- One object per class can be marked as a *companion* object using the `companion object` declaration. It is not given a name. The static methods are accessible through the

containing class.

- Functions defined on a companion object are shared by all class instances.
- An *object expression* uses the `object` keyword to declare an anonymous object on the fly with no predefined type. They are the equivalent of anonymous inner classes in Java.

```
window.addMouseListener(object : MouseAdapter() {
    override fun mouseClicked(e: MouseEvent) {
        // Code that runs when the mouse is clicked
    }

    override fun mouseReleased(e: MouseEvent) {
        // Code that runs when the mouse is released
    }
})
```

Extensions

- Extensions let you add new functions and properties to an existing type.

```
fun Double.toDollar(): String {
    return "$$this"
}
```

- Extensions can be used in place of the **Decorator** pattern.

Return, break, and continue

- `return` returns from the enclosing function.
- `break` terminates the enclosing loop.
- `continue` moves to the next iteration of the enclosing loop.
- You can use labels with break and continue for nested loops.

```
myloop@ while (x < 20) {
    while (y < 20) {
        x++
        break@myloop
    }
}
```

- You can also use labels with `return` for nested functions.

More fun with functions

- `vararg` allows you to specify a function takes a variable number of arguments: `fun <T> valuesToList(vararg vals: T): MutableList<T>`
- The `vararg` function is passed an array of values.
- If you have an array of values you want to pass, use `*`, the *spread operator*. `valuesToList(*myArray)` or even `valuesToList(0, *myArray, 6, 7)`.
- You can declare a function as infix with the `infix` keyword.
- You can also declare a function `inline` for performance.

Interoperability

- This is an interesting topic the book made no useful points on.